



Chapter 11

Database Performance Tuning and Query Optimization



Learning Objectives

- **After completing this chapter, you will be able to:**
 - Identify the procedures involved in database performance tuning
 - Describe how a DBMS processes SQL queries in each of its three phases
 - Explain the role of indexes in speeding up data access
 - Differentiate between a rule-based optimizer and a cost-based optimizer
 - Describe some common practices used to write efficient SQL code
 - Explain how to formulate queries and tune the DBMS for optimal performance



Database Performance-Tuning Concepts

- One of the main functions of a database system is to provide timely answers
 - End users interact with the DBMS through the use of queries to generate information, using the following sequence:
 - 1. End-user (client-end) application generates a query
 - 2. Query is sent to the DBMS (server end)
 - 3. DBMS (server end) executes the query
 - 4. DBMS sends the resulting data set to the end-user (client-end) application



Database Performance-Tuning Concepts

- Goal of database performance is to execute queries as fast as possible
 - **Database performance tuning**: set of activities and procedures that reduce response time of database system, i.e., to ensure that an end-user query is processed by the DBMS in the minimum amount of time
- Fine-tuning the performance of a database system requires a holistic approach
 - All factors must operate at optimum level with minimal bottlenecks



Database Performance-Tuning Concepts

- In general, the performance of a typical DBMS is constrained by **three** main factors:
 - CPU processing power
 - Available primary memory (RAM)
 - Input/output (hard disk and network) throughput
- Table 11.1 on the next slide lists some **system components** and summarizes **general guidelines** for achieving better query performance



Database Performance-Tuning Concepts

TABLE 11.1

GENERAL GUIDELINES FOR BETTER SYSTEM PERFORMANCE

	SYSTEM RESOURCES	CLIENT	SERVER
Hardware	CPU	The fastest possible Dual-core CPU or higher	The fastest possible Multiple processors (quad-core technology) Cluster of networked computers
	RAM	The maximum possible to avoid OS memory to disk swapping	The maximum possible to avoid OS memory to disk swapping
	Hard disk	Fast SATA/EIDE hard disk with sufficient free hard disk space Solid State Drives (SSD) for faster speed	Multiple high-speed, high-capacity disks Fast disk interface (SAS / SCSI / Firewire / Fibre Channel) RAID configuration optimized for throughput Solid State Drives (SSD) for faster speed Separate disks for OS, DBMS, and data spaces
	Network	High-speed connection	High-speed connection
Software	Operating System (OS)	64-bit OS for larger address spaces Fine-tuned for best client application performance	64-bit OS for larger address spaces Fine-tuned for best server application performance
	Network	Fine-tuned for best throughput	Fine-tuned for best throughput
	Application	Optimize SQL in client application	Optimize DBMS server for best performance



Performance Tuning: Client and Server

- Database performance-tuning activities can be divided into those on the **client side** and those on the **server side**.
 - **Client side**: SQL performance tuning
 - Generate SQL query that returns correct answer in **least amount of time**, using minimum amount of resources at server
 - **Server side**: DBMS performance tuning
 - DBMS environment is configured to respond to clients' requests **as fast as possible**, while making optimum use of existing resources



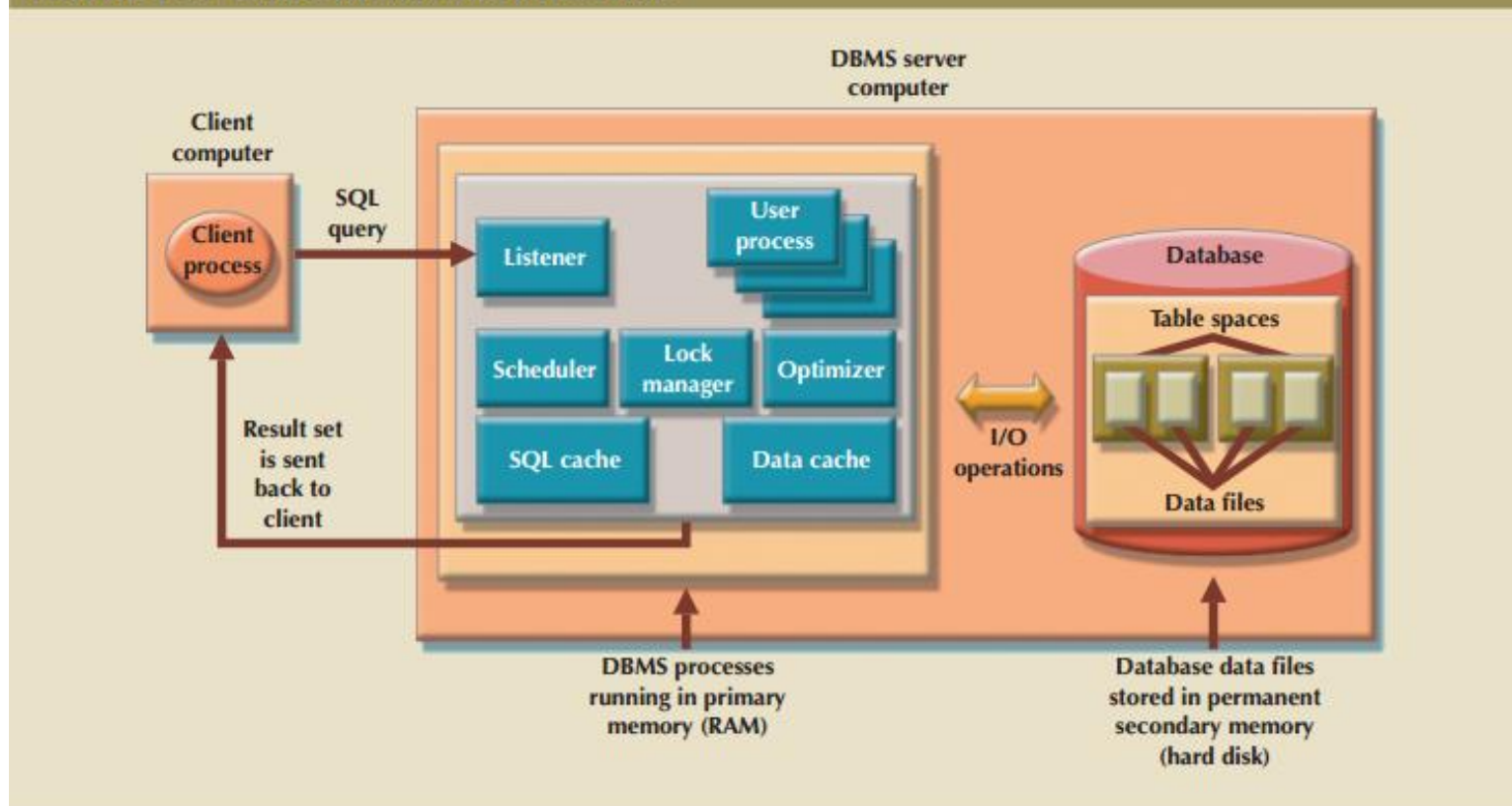
DBMS Architecture

- Architecture of a DBMS is represented by the processes and structures (**in memory and permanent storage**) used to manage a database
 - Such processes collaborate with one another to perform specific functions
- Figure 11.1 on the next slide illustrates the basic DBMS architecture



DBMS Architecture

FIGURE 11.1 BASIC DBMS ARCHITECTURE





DBMS Architecture

- Note the following components and functions in the figure:
 - All data in a database are stored in **data files**
 - **Data files** automatically expand in predefined increments known as **extends**
 - Data files are grouped in **file groups** or **table spaces**
 - **File group/table space** is a logical grouping of several data files that store data with similar characteristics
 - **Data cache or buffer cache**: shared, reserved memory
 - Stores the most recently accessed data blocks in RAM



DBMS Architecture

- **SQL cache or procedure cache**: shared, reserved memory
 - Stores most recently executed SQL statements or PL (procedural language)/SQL procedures, including triggers and functions
- To work with data, DBMS retrieves data from permanent storage and places it in RAM
 - Data is retrieved from the data files and placed in the data cache



DBMS Architecture

- To move data from permanent storage (data files) to RAM (data cache), the DBMS issues I/O requests and waits for the replies.
 - **Input/output request:** low-level data access operation that reads or writes data to and from computer devices, such as memory, hard disks, video, and printers
- Majority of performance-tuning activities focus on minimizing the number of I/O operations
 - I/O operations is many times slower than reading data from the data cache



DBMS Architecture

- **Figure 11.1 also illustrates some typical DBMS processes:**
 - **Listener process** – listens for clients' requests and handles the processing of the SQL requests to other DBMS processes
 - **User process** – handles all requests submitted to the server during each client's session
 - **Scheduler process** – organizes the concurrent execution of SQL requests
 - **Lock manager process** – manages all locks placed on database objects, including disk pages
 - **Optimizer process** – analyses SQL queries and finds the most efficient way to access the data



Database Query Optimization Modes

- Algorithms proposed for query optimization are based on two principles:
 - Selection of the optimum execution order to achieve the fastest execution time
 - Selection of sites to be accessed to minimize communication costs
- Query optimization algorithm can be evaluated on the basis of its:
 - Operation mode, or
 - Timing of its optimization



Database Query Optimization Modes

- Operation modes can be classified as:
 - Automatic query optimization:
 - DBMS finds the most cost-effective access path without user intervention
 - Manual query optimization:
 - Requires that optimization be selected and scheduled by the end-user or programmer
- Automatic query optimization is more desirable from the end user's point of view
 - However, such convenience increases overhead on the DBMS



Database Query Optimization Modes

- Query optimization algorithms can also be classified according to **when the optimization is done**.
- Within this timing classification, query optimization algorithms can be classified as:
 - **Static query optimization:**
 - Takes place at compilation time
 - Best optimization strategy is selected when the query is compiled by the DBMS
 - **Dynamic query optimization:**
 - Takes place at execution time
 - Access strategy is dynamically determined by the DBMS at run time, using the most up-to-date information about the database



Database Query Optimization Modes

- Query optimization techniques can also be classified according to the **type of information that is used to optimize the query**, e.g., queries may be based on **statistically based** or **rule-based algorithms**
- **Statistically based query optimization algorithm:**
 - Uses statistical information about the database, such as **size, number of records, average access time, number of requests serviced, and number of users with access rights**
 - Such statistics are used by the DBMS to determine the best access strategy



Database Query Optimization Modes

- Statistical information is generated by DBMS through:
 - **Dynamic statistical generation mode** – DBMS automatically evaluates and updates the statistics after each data access operation
 - **Manual statistical generation mode** – statistics must be updated periodically through a user-selected utility, such as IBM's RUNSTAT command, which is used by DB2 DBMSs
- **Rule-based query optimization algorithm:**
 - Based on a set of user-defined rules to determine the best query access strategy
 - The rules are entered by the end user or database administrator



Database Statistics

- Refers to a number of measurements about database objects
- Provides a snapshot of database characteristics, such as:
 - Number of processors used
 - Processor speed
 - Temporary space available
- DBMS uses this statistics to make crucial decision about improving query processing efficiency



Database Statistics

- Database statistics can be gathered either:
 - **Manually by the DBA,**
 - **Automatically by the DBMS,**
- E.g.:
 - **ANALYSE command in SQL** to gather statistics provided by many DB vendors,
 - Other vendors have their own routines to gather statistics
- A sample of measurements that the DBMS may gather about various database objects is shown on the next slide



Database Statistics

TABLE 11.2

SAMPLE DATABASE STATISTICS MEASUREMENTS

DATABASE OBJECT	SAMPLE MEASUREMENTS
Tables	Number of rows, number of disk blocks used, row length, number of columns in each row, number of distinct values in each column, maximum value in each column, minimum value in each column, and columns that have indexes
Indexes	Number and name of columns in the index key, number of key values in the index, number of distinct key values in the index key, histogram of key values in an index, and number of disk pages used by the index
Environment Resources	Logical and physical disk block size, location and size of data files, and number of extends per data file



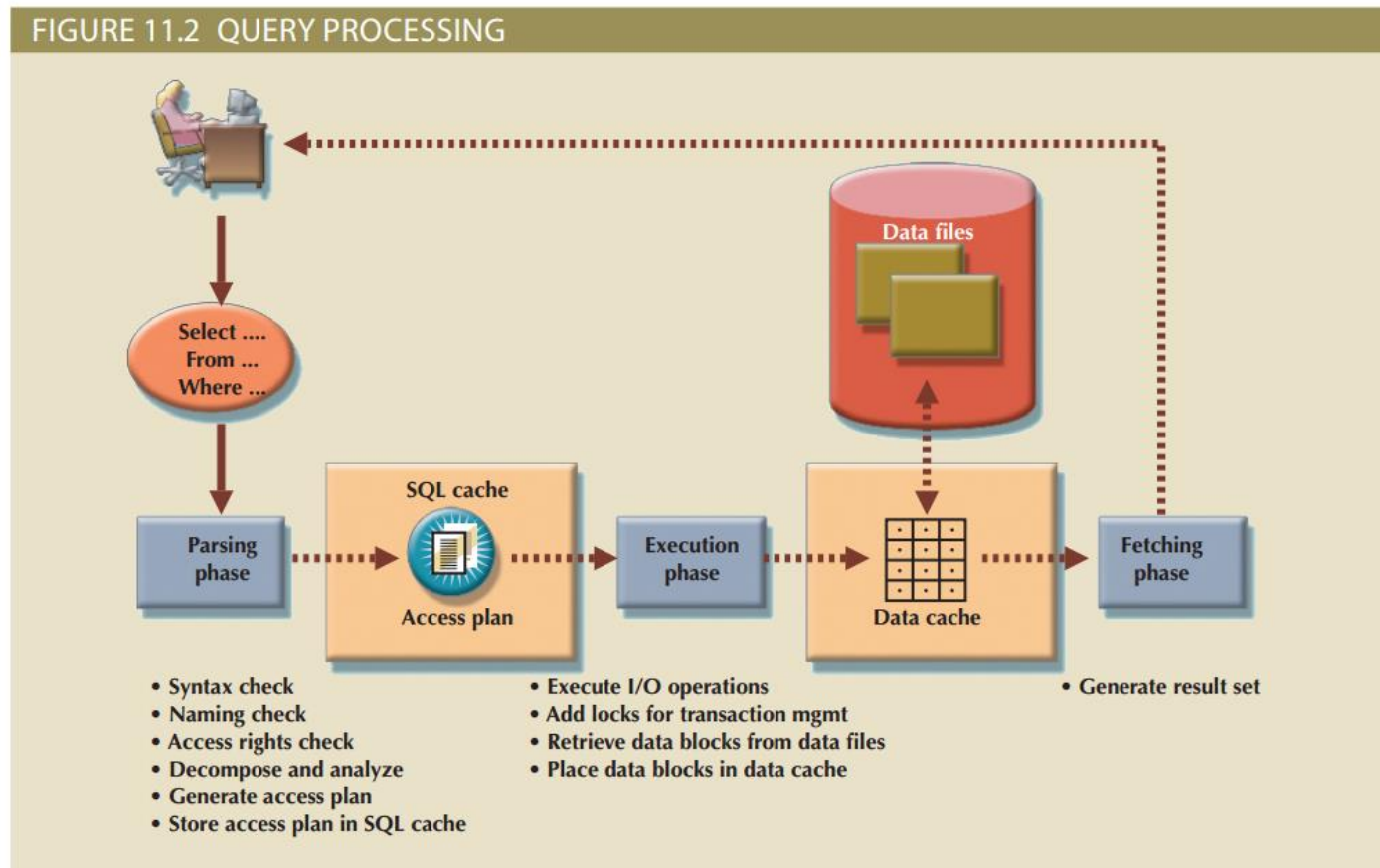
Query Processing

- What happens at the DBMS server end when the client's SQL statement is received?
- DBMS processes a query in 3 phases:
 - **Parsing**
 - DBMS parses the SQL query and chooses the most efficient access/execution plan
 - **Execution**
 - DBMS executes the SQL query using the chosen execution plan
 - **Fetching**
 - DBMS fetches the data and sends the result set back to the client



Query Processing

- Figure 11.2 below shows the general steps required for query processing





SQL Parsing Phase

- **Query is broken down into smaller units**
 - Original SQL query is transformed into slightly different version of original SQL code, which is fully equivalent and more efficient
- The SQL parsing activities are performed by the query optimizer
- **Query optimizer:**
 - Analyzes SQL query and finds the most efficient way to access data
 - Most time-consuming phase in query processing



SQL Parsing Phase

- Parsing a SQL query requires several steps, in which the SQL query is:
 - Validated for syntax compliance
 - Validated against the data dictionary to ensure that table names and column names are correct
 - Validated against the data dictionary to ensure that the user has proper access rights
 - Analyzed and decomposed into more atomic components
 - Optimized through transformation into a fully equivalent but more efficient SQL query
 - Prepared for execution by determining the most efficient execution or access plan



SQL Parsing Phase

- Once the SQL statement is transformed, the DBMS creates what is commonly known as an **access plan** or **execution plan**
- **An access plan is the result of parsing a SQL statement**
 - Contains series of steps a DBMS will use to execute the query and return the result set in the most efficient way
- Access plans are **DBMS-specific** and **translate the client's SQL query into the series of complex I/O operations required to read the data from the physical data files and generate the result set**
- Table 11.3 illustrates some I/O operations for an Oracle RDBMS
 - Most DBMSs perform similar types of operations when accessing and manipulating data sets



SQL Parsing Phase

TABLE 11.3

SAMPLE DBMS ACCESS PLAN I/O OPERATIONS

OPERATION	DESCRIPTION
Table scan (full)	Reads the entire table sequentially, from the first row to the last, one row at a time (slowest)
Table access (row ID)	Reads a table row directly, using the row ID value (fastest)
Index scan (range)	Reads the index first to obtain the row IDs and then accesses the table rows directly (faster than a full table scan)
Index access (unique)	Used when a table has a unique index in a column
Nested loop	Reads and compares a set of values to another set of values, using a nested loop style (slow)
Merge	Merges two data sets (slow)
Sort	Sorts a data set (slow)



SQL Execution Phase

- All I/O operations indicated in the access plan are executed
 - Locks are acquired
 - Data are retrieved from data files and placed in data cache
 - Transaction management commands are processed



SQL Fetching Phase

- Rows of resulting query result set are returned to client
 - DBMS may use temporary table space to store temporary data
 - Database server coordinates the movement of the result set rows from the server cache to the client cache



Query Processing Bottlenecks

- Delay introduced in the processing of an I/O operation that slows the system
- **Within a DBMS, there are five components that typically cause bottlenecks**
 - CPU
 - RAM
 - Hard disk
 - Network
 - Application code



Indexes and Query Optimization

- **Indexes**

- Help speed up data access
 - Facilitate searching, sorting, using aggregate functions and join operations
- Speeding up data access occurs because an index is an ordered set of values that contain the index key and pointers
- An index scan is more efficient than a full table scan;
 - index data is preordered and the amount of data is usually much smaller

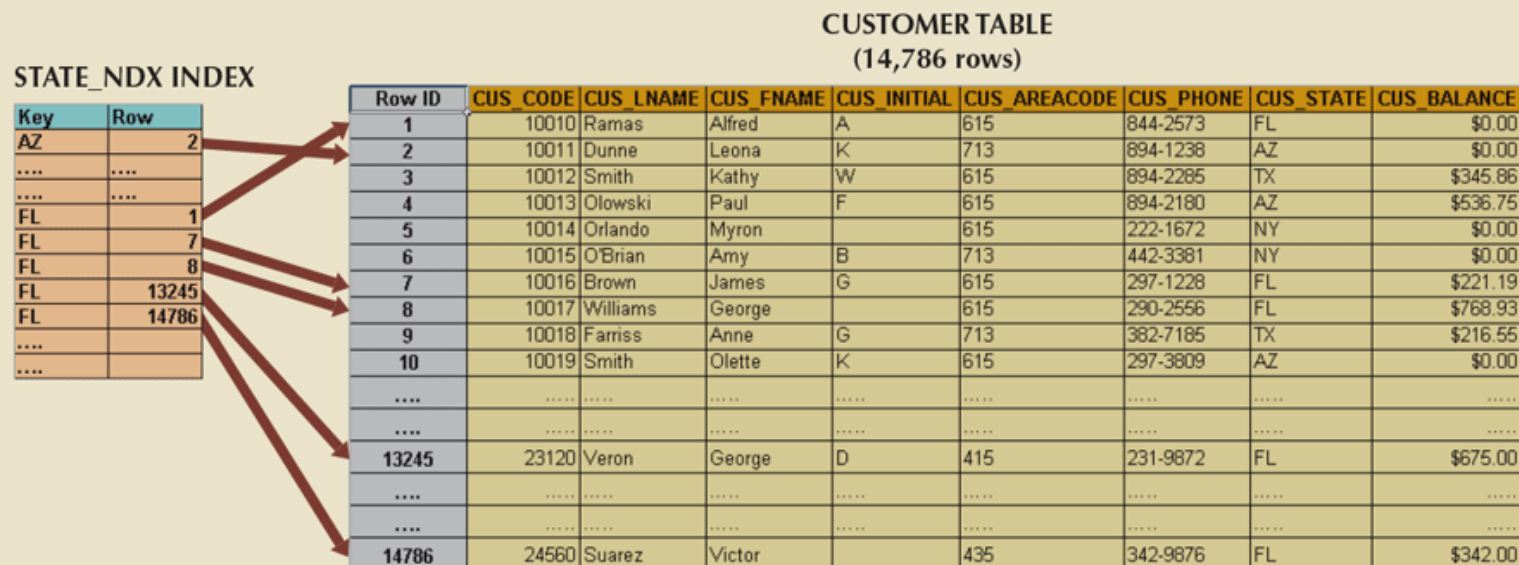


Indexes and Query Optimization

Suppose the following query is submitted to the DBMS:

```
SELECT CUS_LNAME, CUS_STATE  
FROM CUSTOMER  
WHERE CUS_STATE = "FL";
```

FIGURE 11.3 INDEX REPRESENTATION FOR THE CUSTOMER TABLE





Indexes and Query Optimization

- One measure that determines the need for an index is the **data sparsity** of a column.
 - **Data sparsity**: number of different values a column could have
 - Can be high or low sparsity
- Knowing the sparsity helps decide whether the use of an index is appropriate.
 - **E.g.**, when performing a search in a column with low sparsity, you are likely to read a high percentage of the table rows.
 - Therefore, index processing might be unnecessary.



Indexes and Query Optimization

- Data structures used to implement indexes
 - Hash indexes
 - Good for simple and fast lookup operations based on equality conditions, e.g., LNAME="Maimane" and FNAME="Thabiso"
 - B-tree indexes
 - Default and most common type of index used in databases
 - Used in tables in which column values repeat a relatively small number of times
 - Bitmap indexes
 - Used mostly in data warehouse applications in tables with a large number of rows in which a small number of column values repeat many times
- DBMS determines best type of index to use



Optimizer Choices

- Query optimization is the central activity during the parsing phase in query processing
- Query optimizer can operate in one of two modes:
 - **Rule-based optimizer:**
 - Uses preset rules and points to determine the best approach to execute a query
 - Rules assign a “fixed cost” to each SQL operation
 - The costs are then added to yield the cost of the execution plan, e.g., a full table scan has a set cost of 10, while a table access by row ID has a set cost of 3



Optimizer Choices

- **Cost-based optimizer:**
 - Uses sophisticated algorithms based on statistics about objects being accessed to determine the best approach to execute a query
 - Optimizer process adds up the processing cost, I/O costs, and resource costs (RAM and temporary space) to determine the total cost of a given execution plan



Optimizer Example

- **Optimizer's objective:**

- Find alternative ways to execute a query—to evaluate the “cost” of each alternative and then to choose the one with the lowest cost

- **An example to help us understand the function of the query optimizer**

- Assume that we want to list all products provided by a vendor based in Gauteng, to acquire that information, we could write the following query:

```
SELECT P_CODE, P_DESCRIPT, P_PRICE, V_NAME, V_PROVINCE  
FROM PRODUCT, VENDOR  
WHERE PRODUCT.V_CODE = VENDOR.V_CODE  
AND VENDOR.V_PROVINCE = 'GP';
```



Optimizer Example

- Furthermore, let us assume that the database statistics indicate that:
 - The PRODUCT table has 7,000 rows
 - The VENDOR table has 300 rows
 - Ten vendors are located in Gauteng
 - One thousand products come from vendors in Gauteng
- Armed with this information, the optimizer would try to find the most efficient way to access the data. **Note that:**
 - Only the first two items are available to the optimizer
 - The second two items are assumed to illustrate the choices that the optimizer must make



Optimizer Example

- The optimizer would now try to find the most efficient way to access the data
- The primary factor in determining the most efficient access plan is the I/O cost
- The table on the next slide shows two sample access plans for the previous query and their respective I/O costs
- To make the example easier to understand:
 - The I/O Operations and I/O Cost columns in the table estimate only the number of I/O disk reads the DBMS must perform.
 - For simplicity's sake, it is assumed that there are no indexes and that each row read has an I/O cost of 1.



Optimizer Example

COMPARING ACCESS PLANS AND I/O COSTS

PLAN	STEP	OPERATION	I/O OPERATIONS	I/O COST	RESULTING SET ROWS	TOTAL I/O COST
A	A1	Cartesian product (PRODUCT, VENDOR)	7,000 + 300	7,300	2,100,000	7,300
	A2	Select rows in A1 with matching vendor codes	2,100,000	2,100,000	7,000	2,107,300
	A3	Select rows in A2 with V_PROVINCE='GP'	7,000	7,000	1,000	2,114,300
B	B1	Select rows in VENDOR with V_PROVINCE='GP'	300	300	10	300
	B2	Cartesian Product (PRODUCT, B1)	7,000 + 10	7,010	70,000	7,310
	B3	Select rows in B2 with matching vendor codes	70,000	70,000	1,000	77,310



Using Hints to Affect Optimizer Choices

- Optimizer might not choose the best execution plan
 - Makes decisions based on existing statistics; might be old
 - Might choose less-efficient decisions even with current statistics
- Sometimes, the end user may want to change the optimizer mode for the current SQL statement
 - To do that, use **optimizer hints**
 - **Optimizer hints**: special instructions for the optimizer embedded in the SQL command text



Few most common optimizer hints used in SQL

TABLE 11.5

OPTIMIZER HINTS

HINT	USAGE
ALL_ROWS	Instructs the optimizer to minimize the overall execution time—that is, to minimize the time needed to return all rows in the query result set. This hint is generally used for batch mode processes. For example: <pre>SELECT /*+ ALL_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
FIRST_ROWS	Instructs the optimizer to minimize the time needed to process the first set of rows—that is, to minimize the time needed to return only the first set of rows in the query result set. This hint is generally used for interactive mode processes. For example: <pre>SELECT /*+ FIRST_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
INDEX(name)	Forces the optimizer to use the P_QOH_NDX index to process this query. For example: <pre>SELECT /*+ INDEX(P_QOH_NDX) */ * FROM PRODUCT WHERE P_QOH < 10</pre>



SQL Performance Tuning

- Evaluated from client perspective
- Therefore, the goal is to illustrate some common practices used to write efficient SQL code
 - Most current relational DBMSs perform automatic query optimization at the server end
 - Most SQL performance optimization techniques are DBMS-specific and thus rarely portable
- Majority of performance problems are related to poorly written SQL code
 - A carefully written query almost always outperforms a poorly written one



Index Selectivity

- Indexes are the most important technique used in SQL performance optimization
 - The key is to know when an index is used
 - As a general rule, indexes are likely to be used:
 - When an indexed column appears by itself in the search criteria of a WHERE or HAVING clause
 - When an indexed column appears by itself in a GROUP BY or ORDER BY clause
 - When a MAX or MIN function is applied to an indexed column
 - When the data sparsity on the indexed column is high



Index Selectivity

- Indexes are very useful when you want to select a small subset of rows from a large table based on a given condition
- The objective is to create indexes with **high selectivity**
 - **Index selectivity** is a measure of the likelihood that an index will be used in query processing



Index Selectivity

- **Some general guidelines for creating and using indexes:**
 - Create indexes for each single attribute used in a WHERE, HAVING, ORDER BY, or GROUP BY clause
 - Do not use indexes in small tables or tables with low sparsity
 - Declare primary and foreign keys so the optimizer can use the indexes in join operations
 - Declaration of a PK or FK automatically create an index for the declared column
 - Declare indexes in join columns other than PK or FK
 - If you perform join operations on columns other than the primary and foreign keys, you might be better off declaring indexes in those columns



Index Selectivity

- Use of index does not always improve performance
 - Some DBMSs, indexes are ignored when functions are used in the table attributes, e.g., $P_QOH > P_MIN * 1.10$
- However, major databases now support function-based indexes.
 - **Function-based index**: an index based on a specific SQL function or expression
 - Useful when dealing with derived attributes
- How many indexes should you create?
 - Note that too many indexes will slow down INSERT, UPDATE, and DELETE operations, especially if the table contains many thousands of rows



Conditional Expressions

- Normally placed within WHERE or HAVING clauses of an SQL statement
 - Restricts the output of a query to only rows matching conditional criteria
 - The conditional criteria have the form shown in table below

TABLE 11.6

CONDITIONAL CRITERIA

OPERAND1	CONDITIONAL OPERATOR	OPERAND2
P_PRICE	>	10.00
V_STATE	=	FL
V_CONTACT	LIKE	Smith%
P_QOH	>	P_MIN * 1.10



Conditional Expressions

- Guidelines to write efficient conditional expressions in SQL code:
 - Use simple columns or literals as operands, **e.g.**, comparing `P_PRICE > 10.00` is faster than `P_QOH > P_MIN * 1.10`
 - Numeric field comparisons are faster than character, date, and NULL comparisons
 - Equality comparisons are faster than inequality comparisons
 - Transform conditional expressions to use literals, **e.g.**, if your condition is `P_PRICE - 10 = 7`, change it to read `P_PRICE = 17`



Conditional Expressions

- Guidelines to write efficient conditional expressions in SQL code:
 - Write equality conditions first when using multiple conditional expressions, **e.g.**, if you have a composite condition such as `P_QOH < P_MIN AND P_MIN = P_REORDER AND P_QOH = 10`, change it to read `P_QOH = 10 AND P_MIN = P_REORDER AND P_MIN > 10`
 - When using multiple AND conditions, write the condition most likely to be false first
 - When using multiple OR conditions, put the condition most likely to be true first
 - Avoid the use of NOT logical operator - transform an SQL expression containing a NOT logical operator into an equivalent expression, **e.g.**, `NOT (P_PRICE > 10.00)` can be written as `P_PRICE <= 10.00`



Query Formulation

- **Step to formulate a query**
 - Identify what columns and computations are required
 - Identify source tables
 - Decide how to join tables
 - Determine what selection criteria is needed
 - Determine the order in which to display the output



DBMS Performance Tuning

- Includes tasks such as managing DBMS processes in primary memory and the structures in physical storage
 - DBMS performance tuning at server end focuses on setting parameters used for:
 - Data cache
 - SQL cache
 - Sort cache
 - Optimizer mode



DBMS Performance Tuning

- It would be optimal to have the entire database stored in primary memory to minimize costly disk access.
- **In-memory database**: store large portions of the database in primary storage rather than secondary (disk) storage
- These systems are becoming popular because of:
 - Increasing performance demands of modern database applications
 - Diminishing costs
 - Technology advances of components, such as flash-memory and solid state drives



DBMS Performance Tuning

- **Recommendations for physical storage of databases:**
 - Utilize I/O accelerators - uses flash solid state drives (SSDs) to store the database.
 - Use RAID (**Redundant Array of Independent Disks**) to provide a balance between performance improvement and fault tolerance
 - Minimize disk contention by using multiple, independent storage volumes
 - Put high-usage tables in their own table spaces
 - Assign separate data files in separate storage volumes for indexes, system and high-usage tables
 - Take advantage of the various table storage organizations in the database
 - Partition tables based on usage
 - Apply denormalized tables where appropriate
 - Store computed and aggregate attributes in tables



Summary

- Database performance tuning refers to a set of activities and procedures designed to ensure that an end-user query is processed by the DBMS in the least amount of time
- Database statistics refer to a number of measurements gathered by the DBMS that describe a snapshot of the database objects' characteristics
- DBMSs process queries in three phases: parsing, execution, and fetching
- Indexes are crucial in the process that speeds up data access
- During query optimization, the DBMS must choose what indexes to use, how to perform join operations, which table to use first, and so on



Summary

- A rule-based optimizer uses preset rules and points to determine the best approach to execute a query
- SQL performance tuning deals with writing queries that make good use of the statistics
- Query formulation deals with how to translate business questions into specific SQL code to generate the required results
- DBMS performance tuning includes tasks such as managing the DBMS processes in primary memory (allocating memory for caching purposes) and managing the structures in physical storage (allocating space for the data files)